

Modify Dijkstra's algorithm to compute the shortest paths from a given source vertex s in $O(WV + E)$ time.

22.3-10

Modify your algorithm from Exercise 22.3-9 to run in $O((V + E) \lg W)$ time. (*Hint*: How many distinct shortest-path estimates can $V - S$ contain at any point in time?)

22.3-11

Suppose that you are given a weighted, directed graph $G = (V, E)$ in which edges that leave the source vertex s may have negative weights, all other edge weights are nonnegative, and there are no negative-weight cycles. Argue that Dijkstra's algorithm correctly finds shortest paths from s in this graph.

22.3-12

Suppose that you have a weighted directed graph $G = (V, E)$ in which all edge weights are positive real values in the range $[C, 2C]$ for some positive constant C . Modify Dijkstra's algorithm so that it runs in $O(V + E)$ time.

22.4 Difference constraints and shortest paths

Chapter 29 studies the general linear-programming problem, showing how to optimize a linear function subject to a set of linear inequalities. This section investigates a special case of linear programming that reduces to finding shortest paths from a single source. The Bellman-Ford algorithm then solves the resulting single-source shortest-paths problem, thereby also solving the linear-programming problem.

Linear programming

In the general *linear-programming problem*, the input is an $m \times n$ matrix A , an m -vector b , and an n -vector c . The goal is to find a vector x of n elements that maximizes the *objective function* $\sum_{i=1}^n c_i x_i$ subject to the m constraints given by $Ax \leq b$.

The most popular method for solving linear programs is the *simplex algorithm*, which Section 29.1 discusses. Although the simplex algorithm does not always run in time polynomial in the size of its input, there are other linear-programming algorithms that do run in polynomial time. We offer here two reasons to understand the setup of linear-programming problems. First, if you know that you can cast a given problem as a polynomial-sized linear-programming problem, then you immediately have a polynomial-time algorithm to solve the problem. Second, faster algorithms exist for many special cases of linear programming. For example, the single-pair shortest-path problem (Exercise 22.4-4) and the maximum-flow problem (Exercise 24.1-5) are special cases of linear programming.

Sometimes the objective function does not matter: it's enough just to find any *feasible solution*, that is, any vector x that satisfies $Ax \leq b$, or to determine that no feasible solution exists. This section focuses on one such *feasibility problem*.

Systems of difference constraints

In a *system of difference constraints*, each row of the linear-programming matrix A contains one 1 and one -1 , and all other entries of A are 0. Thus, the constraints given by $Ax \leq b$ are a set of *difference constraints* involving n unknowns, in which each constraint is a simple linear inequality of the form

$$x_j - x_i \leq b_k,$$

where $1 \leq i, j \leq n$, $i \neq j$, and $1 \leq k \leq m$.

For example, consider the problem of finding a 5-vector $x = (x_i)$ that satisfies

$$\begin{pmatrix} 1 & -1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & -1 \\ 0 & 1 & 0 & 0 & -1 \\ -1 & 0 & 1 & 0 & 0 \\ -1 & 0 & 0 & 1 & 0 \\ 0 & 0 & -1 & 1 & 0 \\ 0 & 0 & -1 & 0 & 1 \\ 0 & 0 & 0 & -1 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{pmatrix} \leq \begin{pmatrix} 0 \\ -1 \\ 1 \\ 5 \\ 4 \\ -1 \\ -3 \\ -3 \end{pmatrix}.$$

This problem is equivalent to finding values for the unknowns x_1, x_2, x_3, x_4, x_5 , satisfying the following 8 difference constraints:

$$x_1 - x_2 \leq 0, \quad (22.2)$$

$$x_1 - x_5 \leq -1, \quad (22.3)$$

$$x_2 - x_5 \leq 1, \quad (22.4)$$

$$x_3 - x_1 \leq 5, \quad (22.5)$$

$$x_4 - x_1 \leq 4, \quad (22.6)$$

$$x_4 - x_3 \leq -1, \quad (22.7)$$

$$x_5 - x_3 \leq -3, \quad (22.8)$$

$$x_5 - x_4 \leq -3. \quad (22.9)$$

One solution to this problem is $x = (-5, -3, 0, -1, -4)$, which you can verify directly by checking each inequality. In fact, this problem has more than one solution. Another is $x' = (0, 2, 5, 4, 1)$. These two solutions are related: each component of x' is 5 larger than the corresponding component of x . This fact is not mere coincidence.

Lemma 22.8

Let $x = (x_1, x_2, \dots, x_n)$ be a solution to a system $Ax \leq b$ of difference constraints, and let d be any constant. Then $x + d = (x_1 + d, x_2 + d, \dots, x_n + d)$ is a solution to $Ax \leq b$ as well.

Proof For each x_i and x_j , we have $(x_j + d) - (x_i + d) = x_j - x_i$. Thus, if x satisfies $Ax \leq b$, so does $x + d$. ■

Systems of difference constraints occur in various applications. For example, the unknowns x_i might be times at which events are to occur. Each constraint states that at least a certain amount of time, or at most a certain amount of time, must elapse between two events. Perhaps the events are jobs to be performed during the assembly of a product. If the manufacturer applies an adhesive that takes 2 hours to set at time x_1 and has to wait until it sets to install a part at time x_2 , then there is a constraint that $x_2 \geq x_1 + 2$ or, equivalently, that $x_1 - x_2 \leq -2$. Alternatively, the manufacturer might require the part to be installed after the adhesive has been applied but no later than the time that the

adhesive has set halfway. In this case, there is a pair of constraints $x_2 \geq x_1$ and $x_2 \leq x_1 + 1$ or, equivalently, $x_1 - x_2 \leq 0$ and $x_2 - x_1 \leq 1$.

If all the constraints have nonnegative numbers on the right-hand side—that is, if $b_i \geq 0$ for $i = 1, 2, \dots, m$ —then finding a feasible solution is trivial: just set all the unknowns x_i equal to each other. Then all the differences are 0, and every constraint is satisfied. The problem of finding a feasible solution to a system of difference constraints is interesting only if at least one constraint has $b_i < 0$.

Constraint graphs

We can interpret systems of difference constraints from a graph-theoretic point of view. For a system $Ax \leq b$ of difference constraints, let's view the $m \times n$ linear-programming matrix A as the transpose of an incidence matrix (see Exercise 20.1-7) for a graph with n vertices and m edges. Each vertex v_i in the graph, for $i = 1, 2, \dots, n$, corresponds to one of the n unknown variables x_i . Each directed edge in the graph corresponds to one of the m inequalities involving two unknowns.

More formally, given a system $Ax \leq b$ of difference constraints, the corresponding **constraint graph** is a weighted, directed graph $G = (V, E)$, where

$$V = \{v_0, v_1, \dots, v_n\}$$

and

$$E = \{(v_i, v_j) : x_j - x_i \leq b_k \text{ is a constraint}\} \\ \cup \{(v_0, v_1), (v_0, v_2), (v_0, v_3), \dots, (v_0, v_n)\}.$$

The constraint graph includes the additional vertex v_0 , as we shall see shortly, to guarantee that the graph has some vertex that can reach all other vertices. Thus, the vertex set V consists of a vertex v_i for each unknown x_i , plus an additional vertex v_0 . The edge set E contains an edge for each difference constraint, plus an edge (v_0, v_i) for each unknown x_i . If $x_j - x_i \leq b_k$ is a difference constraint, then the weight of

edge (v_i, v_j) is $w(v_i, v_j) = b_k$. The weight of each edge leaving v_0 is 0. Figure 22.8 shows the constraint graph for the system (22.2)–(22.9) of difference constraints.

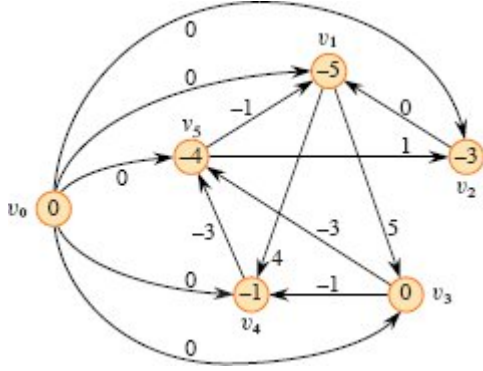


Figure 22.8 The constraint graph corresponding to the system (22.2)–(22.9) of difference constraints. The value of $\delta(v_0, v_i)$ appears in each vertex v_i . One feasible solution to the system is $x = (-5, -3, 0, -1, -4)$.

The following theorem shows how to solve a system of difference constraints by finding shortest-path weights in the corresponding constraint graph.

Theorem 22.9

Given a system $Ax \leq b$ of difference constraints, let $G = (V, E)$ be the corresponding constraint graph. If G contains no negative-weight cycles, then

$$x = (\delta(v_0, v_1), \delta(v_0, v_2), \delta(v_0, v_3), \dots, \delta(v_0, v_n)) \quad (22.10)$$

is a feasible solution for the system. If G contains a negative-weight cycle, then there is no feasible solution for the system.

Proof We first show that if the constraint graph contains no negative-weight cycles, then equation (22.10) gives a feasible solution. Consider any edge $(v_i, v_j) \in E$. The triangle inequality implies that $\delta(v_0, v_j) \leq \delta(v_0, v_i) + w(v_i, v_j)$, which is equivalent to $\delta(v_0, v_j) - \delta(v_0, v_i) \leq w(v_i, v_j)$. Thus, letting $x_i = \delta(v_0, v_i)$ and $x_j = \delta(v_0, v_j)$ satisfies the difference constraint $x_j - x_i \leq w(v_i, v_j)$ that corresponds to edge (v_i, v_j) .

Now we show that if the constraint graph contains a negative-weight cycle, then the system of difference constraints has no feasible solution. Without loss of generality, let the negative-weight cycle be $c = \langle v_1, v_2, \dots, v_k \rangle$, where $v_1 = v_k$. (The vertex v_0 cannot be on cycle c , because it has no entering edges.) Cycle c corresponds to the following difference constraints:

$$\begin{aligned} x_2 - x_1 &\leq w(v_1, v_2), \\ x_3 - x_2 &\leq w(v_2, v_3), \\ &\vdots \\ x_{k-1} - x_{k-2} &\leq w(v_{k-2}, v_{k-1}), \\ x_k - x_{k-1} &\leq w(v_{k-1}, v_k). \end{aligned}$$

We'll assume that x has a solution satisfying each of these k inequalities and then derive a contradiction. The solution must also satisfy the inequality that results from summing the k inequalities together. In summing the left-hand sides, each unknown x_i is added in once and subtracted out once (remember that $v_1 = v_k$ implies $x_1 = x_k$), so that the left-hand side sums to 0. The right-hand side sums to the weight $w(c)$ of the cycle, giving $0 \leq w(c)$. But since c is a negative-weight cycle, $w(c) < 0$, and we obtain the contradiction that $0 \leq w(c) < 0$. ■

Solving systems of difference constraints

Theorem 22.9 suggests how to use the Bellman-Ford algorithm to solve a system of difference constraints. Because the constraint graph contains edges from the source vertex v_0 to all other vertices, any negative-weight cycle in the constraint graph is reachable from v_0 . If the Bellman-Ford algorithm returns TRUE, then the shortest-path weights give a feasible solution to the system. In Figure 22.8, for example, the shortest-path weights provide the feasible solution $x = (-5, -3, 0, -1, -4)$, and by Lemma 22.8, $x = (d - 5, d - 3, d, d - 1, d - 4)$ is also a feasible solution for any constant d . If the Bellman-Ford

algorithm returns FALSE, there is no feasible solution to the system of difference constraints.

A system of difference constraints with m constraints on n unknowns produces a graph with $n + 1$ vertices and $n + m$ edges. Thus, the Bellman-Ford algorithm provides a way to solve the system in $O((n + 1)(n + m)) = O(n^2 + nm)$ time. Exercise 22.4-5 asks you to modify the algorithm to run in $O(nm)$ time, even if m is much less than n .

Exercises

22.4-1

Find a feasible solution or determine that no feasible solution exists for the following system of difference constraints:

$$x_1 - x_2 \leq 1,$$

$$x_1 - x_4 \leq -4,$$

$$x_2 - x_3 \leq 2,$$

$$x_2 - x_5 \leq 7,$$

$$x_2 - x_6 \leq 5,$$

$$x_3 - x_6 \leq 10,$$

$$x_4 - x_2 \leq 2,$$

$$x_5 - x_1 \leq -1,$$

$$x_5 - x_4 \leq 3,$$

$$x_6 - x_3 \leq -8.$$

22.4-2

Find a feasible solution or determine that no feasible solution exists for the following system of difference constraints:

$$x_1 - x_2 \leq 4,$$

$$x_1 - x_5 \leq 5,$$

$$x_2 - x_4 \leq -6,$$

$$x_3 - x_2 \leq 1,$$

$$x_4 - x_1 \leq 3,$$

$$x_4 - x_3 \leq 5,$$

$$x_4 - x_5 \leq 10,$$

$$x_5 - x_3 \leq -4,$$

$$x_5 - x_4 \leq -8.$$

22.4-3

Can any shortest-path weight from the new vertex v_0 in a constraint graph be positive? Explain.

22.4-4

Express the single-pair shortest-path problem as a linear program.

22.4-5

Show how to modify the Bellman-Ford algorithm slightly so that when using it to solve a system of difference constraints with m inequalities on n unknowns, the running time is $O(nm)$.

22.4-6

Consider adding *equality constraints* of the form $x_i = x_j + b_k$ to a system of difference constraints. Show how to solve this variety of constraint system.

22.4-7

Show how to solve a system of difference constraints by a Bellman-Ford-like algorithm that runs on a constraint graph without the extra vertex v_0 .

★ 22.4-8

Let $Ax \leq b$ be a system of m difference constraints in n unknowns. Show that the Bellman-Ford algorithm, when run on the corresponding constraint graph, maximizes $\sum_{i=1}^n x_i$ subject to $Ax \leq b$ and $x_i \leq 0$ for all x_i .

★ 22.4-9

Show that the Bellman-Ford algorithm, when run on the constraint graph for a system $Ax \leq b$ of difference constraints, minimizes the quantity $(\max \{x_i\} - \min \{x_i\})$ subject to $Ax \leq b$. Explain how this fact might come in handy if the algorithm is used to schedule construction jobs.

22.4-10

Suppose that every row in the matrix A of a linear program $Ax \leq b$ corresponds to a difference constraint, a single-variable constraint of the form $x_i \leq b_k$, or a single-variable constraint of the form $-x_i \leq b_k$. Show how to adapt the Bellman-Ford algorithm to solve this variety of constraint system.

22.4-11

Give an efficient algorithm to solve a system $Ax \leq b$ of difference constraints when all of the elements of b are real-valued and all of the unknowns x_i must be integers.

★ 22.4-12

Give an efficient algorithm to solve a system $Ax \leq b$ of difference constraints when all of the elements of b are real-valued and a specified subset of some, but not necessarily all, of the unknowns x_i must be integers.

22.5 Proofs of shortest-paths properties

Throughout this chapter, our correctness arguments have relied on the triangle inequality, upper-bound property, no-path property, convergence property, path-relaxation property, and predecessor-subgraph property. We stated these properties without proof on page 611. In this section, we prove them.

The triangle inequality

In studying breadth-first search (Section 20.2), we proved as Lemma 20.1 a simple property of shortest distances in unweighted graphs. The